

SN1-F25 Practise Midterm Solutions

Keep in mind that there are many correct ways to approach these problems: your solutions do not have to match these ones exactly to get full marks, and it's possible to get part marks for partially correct solutions.

The best way to check your work is to use Pycharm to write your code and check that the output matches what is required.

Solution 1 - Never tell me the odds!

Part 1

```
def never_tell_me_the_odds():  
    for num in range(11, 112):  
        if num % 2 != 0:  
            print(num)  
  
never_tell_me_the_odds()
```

Make sure you understand the [range\(\) function](#) and the [% operator](#). See the links for relevant parts of the course notes.

Part 2

```
def never_tell_me_the_odds(start, end):  
    for num in range(start, end+1):  
        if num % 2 != 0:  
            print(num)  
  
never_tell_me_the_odds(11, 111)  
never_tell_me_the_odds(6, 22)  
never_tell_me_the_odds(-11, 41)
```

Make sure you understand [how to write and use custom functions](#)

Solution 2 - Study Tracker

Part 1

```
total_hours_studied = 0
for hour in [2.1, 1.0, 0.74, 3.2, 2.2, 4.0, 0.15]:
    total_hours += hour
print(f"{total_hours_studied:.2f}")
```

Make sure you understand [the += operator](#). Note that the following two lines of code are equivalent:

```
total_hours += hour

total_hours = total_hours + hour
```

Part 2

```
total_hours_studied = 0
for hour in [2.1, 1.0, 0.74, 3.2, 2.2, 4.0, 0.15]:
    total_hours += hour

if total_hours_studied == 0:
    print(f"You didn't study at all!")
elif total_hours_studied < 10:
    print(f"You studied {total_hours_studied:.2f} hours: nice!")
elif total_hours_studied < 20:
    print(f"You studied {total_hours_studied:.2f} hours: pretty good!")
elif total_hours_studied < 40:
    print(f"You studied {total_hours_studied:.2f} hours: take it easy...")
else:
    print(f"You studied {total_hours_studied:.2f} hours: that's too much!!!")
```

Make sure you understand:

- [conditional statements using if](#)
- [conditional statements using if/elif/else](#)
- [comparison operators and logical operators](#)

- **NOTE:** these are two names for the same concept – confusingly, we have two separate pages in the notes for them. Don't worry about it!

Solution 3 - How Much Things Cost

I've asked the question in an indirect way: given S, G, T , `raw_cost` is supposed to return x . We'll need to modify the formula we've been provided:

$$S = 2.3Gx + 0.42T$$
$$x = \frac{S - 0.42T}{2.3G}$$

Now, let's implement `raw_cost` :

```
def raw_cost(selling_price, grocer_markup, transportation_distance):  
    return (selling_price - 0.42*transportation_distance)/(2.3*G)
```

Part 2

One possible solution:

```
def raw_cost(selling_price, grocer_markup, transportation_distance):  
    alpha = 0.42  
    if transportation_distance < 100:  
        alpha = 0.42  
    elif 100 < transportation_distance < 250:  
        alpha = 0.84  
    elif transportation_distance > 500:  
        alpha = 1.23  
    return (selling_price - alpha*transportation_distance)/(2.3*G)
```

Note: The previous solution can be simplified . For e.g.:

```
def raw_cost(selling_price, grocer_markup, transportation_distance):  
    alpha = 0.42  
    if 100 < transportation_distance < 250:  
        alpha = 0.84  
    elif transportation_distance > 500:  
        alpha = 1.23  
    return (selling_price - alpha*transportation_distance)/(2.3*G)
```

Note:

- simplifying the solution is not required, any solution that *works* is worth full marks
- The way I asked the question, there was no “defined” value of α for T between 250 and 500, also for $T == 100$. Thus, any result for α would be correct for that range. In my solution, I give $\alpha=0.42$ as a default, but that wouldn't have to be your answer for full marks in this case.

Solution 4 - The Burrito Brothers

There are a couple ways to do this problem! I'll give two examples:

1. Using `if/elif/else` alone:

```
def total_burrito_cost(n):
    standard_price = 12
    bulk_price = 6
    bonus_price = 1.11
    after_bonus_price = 11

    total = 0
    if 1 <= n <= 2:
        total += n * standard_price

    elif 2 < n < 11:
        total += 2 * standard_price
        total += (n-2)*bulk_price

    elif n == 11:
        total += 2 * standard_price
        total += 8 * bulk_price
        total += 1 * bonus_price

    elif n > 11:
        total += 2 * standard_price
        total += 8 * bulk_price
        total += 1 * bonus_price
        total += (n-11) * after_bonus_price

    return total

print(total_burrito_cost(20))
```

2. Using if/elif/else with for :

```
def total_burrito_cost(n):
    standard_price = 12
    bulk_price = 6
    bonus_price = 1.11
    after_bonus_price = 11

    total = 0
    for burrito in range(1, n+1):
        if 1 <= burrito <= 2:
            total += standard_price

        elif 2 < burrito < 11:
            total += bulk_price

        elif burrito == 11:
            total += bonus_price

        elif burrito > 11:
            total += after_bonus_price

    return total

print(total_burrito_cost(20))
```

If your implementation is different than the solution, try out a few more test cases to see if your output matches:

```
print(total_burrito_cost(0)) # Output: 0
print(total_burrito_cost(1)) # Output: 12
print(total_burrito_cost(2)) # Output: 24
print(total_burrito_cost(3)) # Output: 30
print(total_burrito_cost(10)) # Output 72
print(total_burrito_cost(11)) # Output 73.11
print(total_burrito_cost(12)) # Output 84.11
print(total_burrito_cost(20)) # Output 172.11
```

Solution 5 - A Time for Turtles

Part 1

There are many ways to do this – the important part is to make sure the turtle **ONLY** draws lines between the given coordinates.

If your solution is different than mine, try both out in Pycharm and see if the turtles draw a square in the same spot.

```
import turtle

turtle.pencolor("red")
turtle.teleport(100, 100)
turtle.setheading(0)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)

turtle.mainloop()
```

Part 2

```
def turtle_square(n):
    turtle.setheading(0)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)

# Don't forget to start the turtle in the right spot with the right color
turtle.pencolor("red")
turtle.teleport(100, 100)

turtle_square(200)
turtle_square(300)
turtle_square(400)
```

Part 3

There's quite a few possible solutions for this. I'll show you just one:

```

def turtle_centered_square(n):
    # Start the turtle on the bottom-left corner as we have done before
    # Make sure the turtle doesn't draw on its way there!
    turtle.up()
    turtle.setheading(0)
    turtle.backward(n/2)
    turtle.right(90)
    turtle.forward(n/2)
    turtle.down()

    # now that we're on the left corner, the problem is the same as before:
    turtle.setheading(0)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)
    turtle.left(90)
    turtle.forward(n)

    # Return turtle to position it started in
    turtle.up()
    turtle.setheading(0)
    turtle.forward(n/2)
    turtle.left(90)
    turtle.forward(n/2)
    turtle.down()

    # Don't forget to start the turtle in the right spot with the right color
    turtle.pencolor("red")
    turtle.teleport(-160, -160)

    turtle_centered_square(80)
    turtle_centered_square(40)
    turtle_centered_square(20)

```

(Optional) Notice that we're repeating some behavior from a previous function... we can simplify by reusing that function!

```

def turtle_centered_square(n):
    # start the turtle on the bottom-left corner as we have done before
    # make sure the turtle doesn't draw on its way there!
    turtle.up()
    turtle.setheading(0)
    turtle.backward(n/2)
    turtle.right(90)

```

```

turtle.forward(n/2)
turtle.down()

# now that we're on the left corner, the problem is the same as before:
# let's reuse the turtle_square function!
turtle_square(n)

# return turtle to position it started in
turtle.up()
turtle.setheading(0)
turtle.forward(n/2)
turtle.left(90)
turtle.forward(n/2)
turtle.down()

# don't forget to start the turtle in the right spot with the right color
turtle.pencolor("red")
turtle.teleport(-160, -160)

turtle_centered_square(80)
turtle_centered_square(40)
turtle_centered_square(20)

```

You can see what the output of both parts of this question should be in the picture below:

